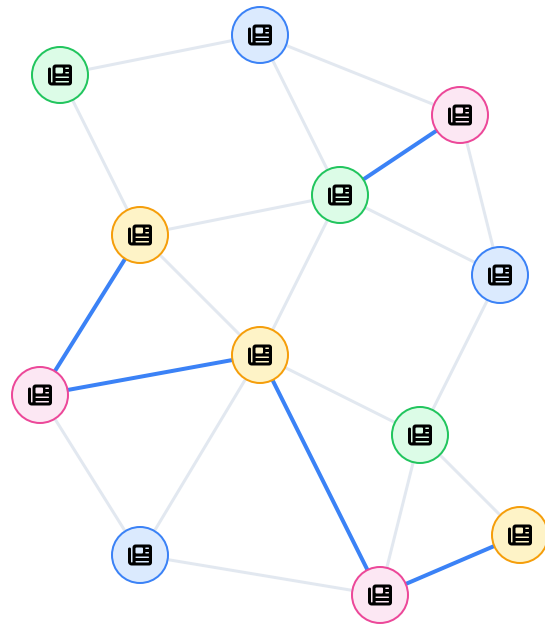


Classificador de Notícias por Grafo

Uma aplicação de grafos, conjuntos, tabelas hash, fila e Busca em Largura para classificação textual.

Universidade de Brasília • Estrutura de Dados

Danilo Sarmento • João Vitor • Kauã Richard • Mário Vinícius • Nathan Pontes



🔗 notícias conectadas por palavras em comum

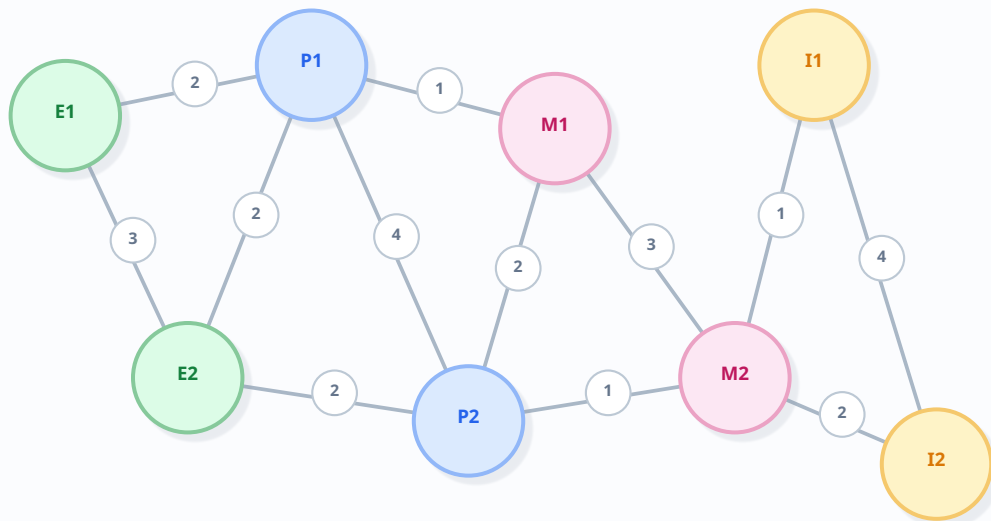
Qual problema o projeto resolve?

Classificar uma notícia nova usando a similaridade com textos já conhecidos.



A ideia central: transformar notícias em um grafo

Cada texto vira um vértice; palavras compartilhadas geram arestas ponderadas.



peso = quantidade de palavras em comum



Modelagem

Vértice

uma notícia da base

Aresta

ligação não direcionada

Peso

nº de tokens compartilhados

Rótulo

categoria conhecida do texto

Mais peso $\Rightarrow$ mais similaridade lexical

Como os arquivos se conectam

Cada módulo possui uma responsabilidade específica e o main.py coordena o fluxo.



dados/noticias.json

base de treinamento com identificador, texto e categoria de cada notícia

160 notícias no relatório

Do texto bruto ao conjunto de palavras relevantes

A função tokenizar reduz ruído antes de comparar as notícias.



Transformação do texto

Entrada

"Presidente sanciona nova lei aprovada pelo congresso!"

minúsculas • regex • split



stopwords • tamanho > 2

Saída: set de tokens

{ presidente, sanciona, nova, lei, aprovada, congresso }

preprocessamento.py

```
1 def tokenizar(texto):
2     texto = texto.lower()
3     texto = re.sub(r"[^a-zà-ú\s]", " ", texto)
4     palavras = texto.split()
5     return {p for p in palavras
6             if p not in STOPWORDS and len(p) > 2}
```



Por que usar set?

Evita repetições e permite calcular a interseção com o operador &.

Duas tabelas hash sustentam a representação

O grafo usa dicionários para guardar dados dos vértices e a lista de adjacência.

self.vertices

idTexto → dados do vértice

not_01 palavras: {futebol, time, gol} **categoria: esporte**

not_02 palavras: {lei, congresso} **categoria: politica**

not_03 palavras: {coleção, desfile} **categoria: moda**

Acesso médio por identificador: $O(1)$

self.adjacencia

idTexto → { idVizinho: peso }

lista de adjacência

```
1 self.adjacencia = {  
2   "not_01": {"not_02": 1, "not_04": 3},  
3   "not_02": {"not_01": 1, "not_05": 2}  
4 }
```

A aresta é registrada nos dois sentidos:



Como o peso de uma aresta é calculado

A similaridade lexical é o tamanho da interseção entre dois conjuntos de tokens.

Exemplo

Notícia A

{ futebol, campeonato, nacional }

Notícia B

{ futebol, time, campeonato }

Interseção

{ futebol, campeonato }

2
peso

grafo.py

```
1 def calcularPeso(self, palavrasA, palavrasB):  
2     return len(palavrasA & palavrasB)  
3  
4 def adicionarAresta(self, idA, idB, peso):  
5     if peso <= 0:  
6         return  
7     self.adjacencia[idA][idB] = peso  
8     self.adjacencia[idB][idA] = peso
```



Arestas com peso zero não são armazenadas; isso evita ligações entre textos sem qualquer termo em comum.

Montagem do grafo de treinamento

Cada notícia é tokenizada, adicionada e conectada aos vértices já existentes.

main.py

```
1 def construirGrafoTreino(noticias):
2     grafo = Grafo()
3
4     for noticia in noticias:
5         palavras = tokenizar(noticia["texto"])
6         grafo.adicionarVertice(
7             noticia["id"], palavras, noticia["categoria"]
8         )
9         grafo.conectarATodos(noticia["id"])
10
11     return grafo
```



Construção incremental



Tokenizar

gera o conjunto de palavras relevantes



Adicionar vértice

salva tokens e categoria em vertices



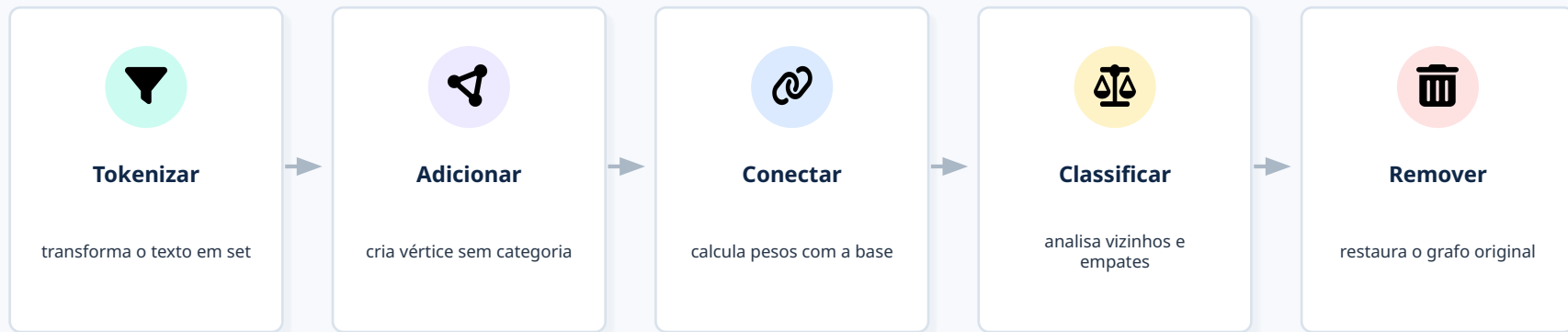
Conectar a todos

compara com os outros textos e cria arestas

Ao final: um grafo com todas as notícias rotuladas

A nova notícia entra no grafo apenas temporariamente

Depois da decisão, o vértice e todas as suas arestas são removidos.



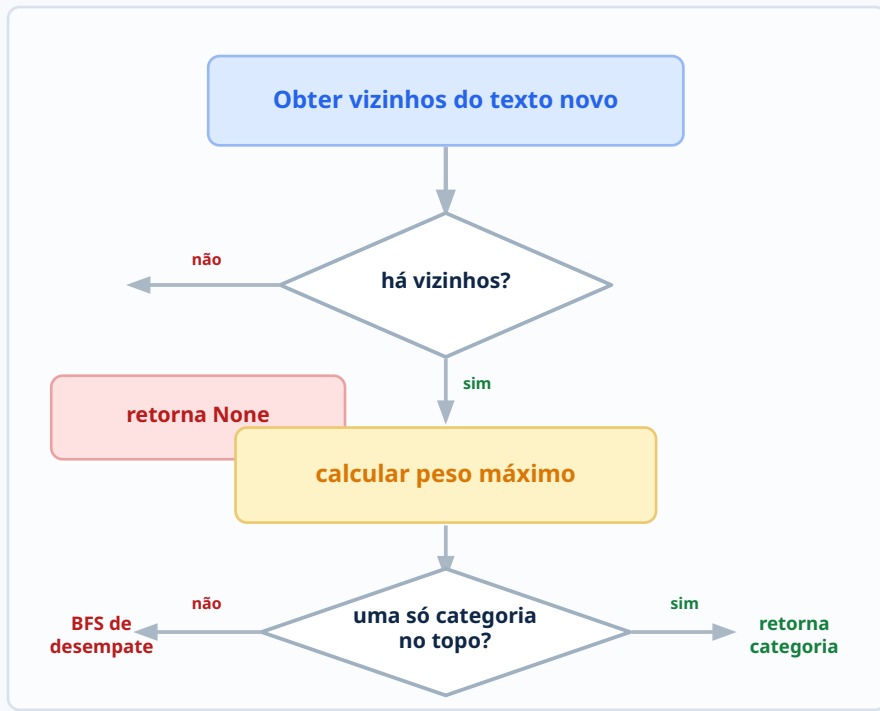
`classificarTextoNovo`

```
1 palavras = tokenizar(texto)
2 grafo.adicionarVertice(idTemporario, palavras, None)
3 grafo.conectarATodos(idTemporario)
4
5 categoriaPrevista = classificador.classificar(idTemporario)
6
7 grafo.removerVertice(idTemporario)
```

A base de treinamento permanece inalterada

Primeira decisão: maior peso entre os vizinhos

O algoritmo verifica quais categorias aparecem na similaridade máxima.

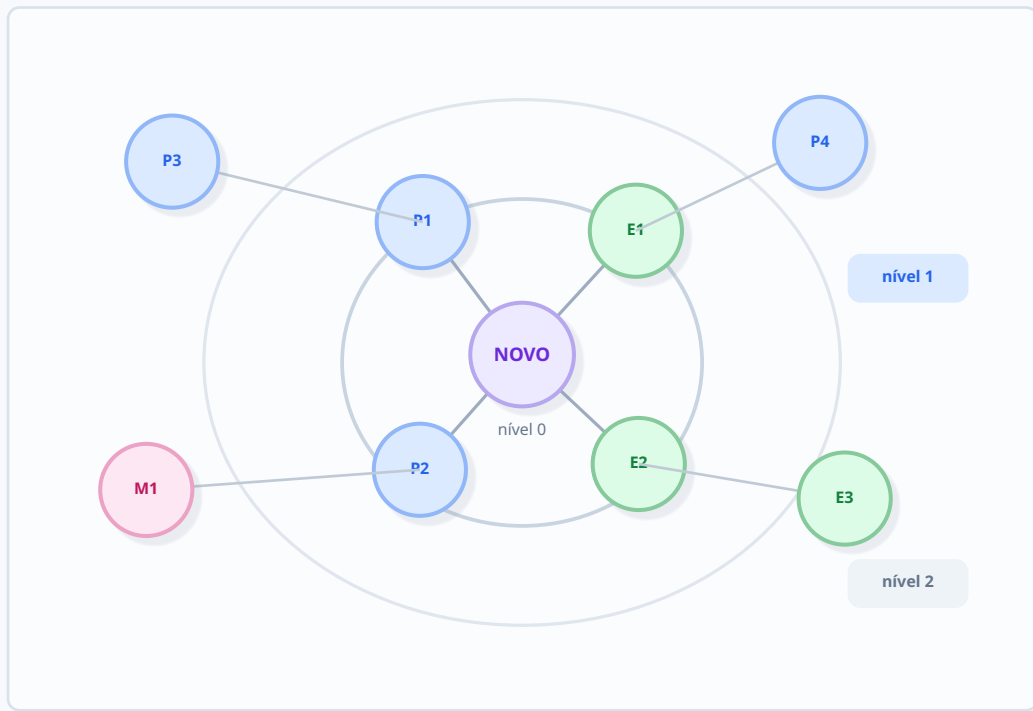


`classificador.py`

```
1 vizinhos = self.grafo.vizinhos(idTextoNovo)
2 if not vizinhos:
3     return None
4
5 pesoMaximo = max(vizinhos.values())
6 categoriasNoTopo = set()
7
8 for vizinhoId, peso in vizinhos.items():
9     if peso == pesoMaximo:
10         categoria = self.grafo.vertices[vizinhoId]["categoria"]
11         categoriasNoTopo.add(categoria)
12
13 if len(categoriasNoTopo) == 1:
14     return categoriasNoTopo.pop()
```

BFS por níveis: os vértices mais próximos têm prioridade

O nível 2 só é considerado se o nível 1 não produzir um vencedor.



Como a BFS decide

- 1 Fila começa com (origem, 0)
- 2 Visitados impede repetições
- 3 Conta categorias do nível atual
- 4 Ao mudar de nível, tenta decidir
- 5 Para após o limite de 2 níveis

Apenas categorias candidatas são comparadas

A fila garante a ordem correta da BFS

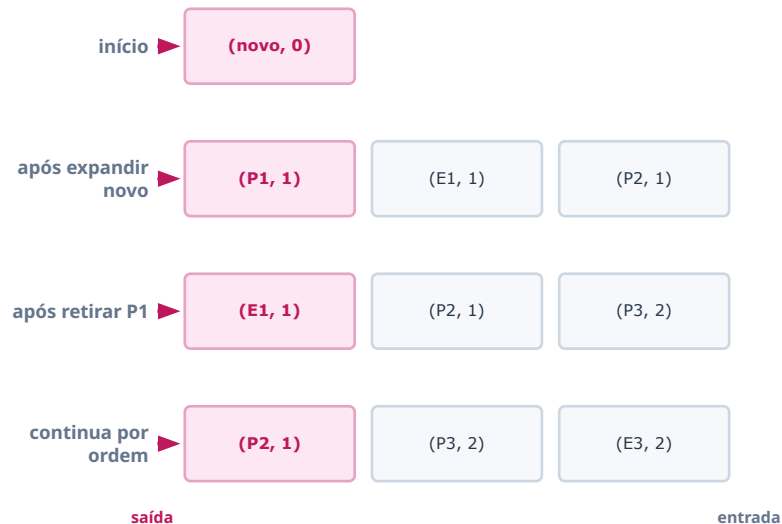
O primeiro elemento inserido é o primeiro a ser removido — comportamento FIFO.

fila.py

```
1 class Fila:
2     def __init__(self):
3         self.dados = []
4         self.inicio = 0
5
6     def enfileirar(self, item):
7         self.dados.append(item)
8
9     def desenfileirar(self):
10        if self.vazia():
11            raise IndexError("Fila vazia.")
12        item = self.dados[self.inicio]
13        self.inicio += 1
14        return item
```



Evolução da fila



Por que “campeonato nacional futebol reforma lei congresso” vira Política?

O maior peso empata entre Esporte e Política; a BFS resolve a ambiguidade.

Texto novo

campeonato nacional futebol reforma lei congresso

6 tokens



Empate no maior peso

ESPORTE

campeonato • futebol • nacional

×

POLÍTICA

congresso • lei • nacional

3

3



BFS no nível 1

Política

27

Esporte

20

Resultado: POLÍTICA

Peso máximo empatado → BFS conta categorias próximas → Política possui maioria → classificação final

Como o programa aparece no terminal

Os testes mostram a decisão direta, o caso indefinido e o desempate por BFS.

```
$ python main.py
Grafo construído com 160 textos de treino.

Notícia: congresso aprova nova lei nacional
-> Categoria prevista: POLITICA

Notícia: campeonato nacional futebol reforma lei
congresso
-> Categoria prevista: POLITICA

Notícia: xylophone quantum banana
-> Categoria prevista: INDEFINIDA
```

Leitura dos exemplos

1

Decisão direta

Quando uma categoria aparece sozinha no maior peso, ela é retornada imediatamente.

2

Empate resolvido

No texto misto, Esporte e Política podem empatar no peso máximo; a BFS decide pela vizinhança.

3

Sem evidência

Se não houver arestas com a base, o classificador retorna None e a saída vira INDEFINIDA.

Entrada → tokenização → grafo temporário → categoria

Resultados dos testes

O desempenho depende da quantidade e da qualidade das palavras em comum com a base.

Cenários observados

Entrada resumida	Situação	Saída	Leitura
congresso • lei • nacional	vocabulário forte de política	POLÍTICA	boa
campeonato • futebol • time	vocabulário forte de esporte	ESPORTE	boa
campeonato • lei	texto curto e ambíguo	depende da BFS	frágil
termos sem relação com a base	sem vizinhos	INDEFINIDA	esperado

Interpretação



Funciona melhor

quando a notícia usa termos característicos de uma categoria.



Fica ambíguo

quando o texto mistura vocabulários de áreas diferentes.



Não “entende” semântica

sinônimos e contexto não são reconhecidos se as palavras forem diferentes.

Resultado principal: o grafo resolve bem casos com evidência lexical; casos curtos dependem do desempate.

Uso de LLM e GitHub

A IA apoiou organização, revisão e explicação; a implementação principal permaneceu do grupo.

Apoio dos LLMs



Uso responsável

LLMs foram usadas como ferramentas de apoio para entendimento, refinamento e sugestões que foram devidamente analisadas pelo grupo de forma responsável.

LLM como apoio, não como substituto da modelagem.

O que foi mantido como trabalho do grupo

Modelagem das notícias como vértices de um grafo;
implementação de lista de adjacência, pesos, fila e BFS;
definição dos critérios de classificação e desempate;
execução, testes manuais e explicação final do código.

Repositório do código-fonte

github.com/Vt010/Classificador-de-noticias-

Hospeda o código em Python, dados e README do projeto.

12. ANÁLISE TÉCNICA

Custos computacionais e decisões de projeto

O modelo é simples e didático, mas pode produzir um grafo bastante denso.



Tokenização

$$O(L)$$

L = tamanho do texto



Peso de uma aresta

$$O(\min(a,b))$$

interseção entre sets



Construção do grafo

$$\approx O(n^2 \cdot k)$$

compara pares de notícias



BFS em empate

$$O(V + E)$$

no subgrafo percorrido



Pontos fortes

- Aplica várias estruturas da disciplina em um único problema;
- É modular, determinístico e fácil de inspecionar;
- Mantém a base intacta após cada consulta.



Limitações

- Considera apenas palavras exatamente iguais;
- A quantidade de arestas pode crescer quadraticamente;
- O desempate alfabético final é arbitrário.

13. EVOLUÇÃO

Como o classificador poderia ser melhorado

As próximas versões podem aumentar a qualidade sem abandonar a modelagem em grafo.



Similaridade normalizada

Substituir a contagem bruta por Jaccard ou cosseno para reduzir o efeito do tamanho dos textos.



Pré-processamento linguístico

Aplicar stemming ou lematização e tratar sinônimos, plurais e variações de palavras.



Controle de densidade

Criar arestas apenas acima de um limiar ou manter somente os k vizinhos mais semelhantes.



Avaliação objetiva

Separar treino e teste e medir acurácia, precisão, revocação e matriz de confusão.

CONCLUSÃO

O projeto transforma um problema de texto em um problema de grafo.

A classificação combina similaridade lexical, arestas ponderadas e uma BFS por níveis para resolver empates.

✓ Grafo • lista de adjacência • set • tabela hash • fila • BFS

Perguntas?

